SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Grama Jyothi Billing Program

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Design, Implement and Validate a Reliable Billing Program
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

TASK 6 – Read and Print Data Correctly

Aim

To read and print each data type with the right specifier and validate the input.

Type	scanf	printf
int	%d	%d
unsigned int / unsigned short	%u / %hu	%u / %hu
long / long long	%ld / %lld	%ld / %lld
float / double	%f / %lf	%f (also for double) / %lf
char / string	%c / %s	%c / %s

Wrong specifier test: printing long v = 5000000000L with %d warns at compile time (-Wformat: "format %d expects int, but argument has type long int") and prints a wrong value; casting (int)v gives 705032704, because only the low 32 bits remain.

Input validation: scanf returns the number of items converted. If r != 3 the line is invalid (e.g. "abc 5 6"); the bad line is discarded with a getchar() loop so that the program does not loop forever.

```
int r = scanf("%d %d %d", &house, &prev, &curr);
if (r == EOF) break;
if (r != 3) { puts("Invalid input"); /* flush line */ continue; }
```

Printing paise as rupees: printf("Rs %ld.%02ld", p/100, p%100); e.g. p = 16250 prints Rs 162.50. (Use %02ld so 5 paise prints .05.)

Programiz
C Online Compiler

main.c

Clear Run

```c

#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
    long p = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS)
    {
        p += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
        units = SLAB2_UNITS;
    }

    if (units > SLAB1_UNITS)
```

Output

```
1 0 150
2 100 200
House: 1
Units: 150
Bill: Rs 162.50
House: 2
Units: 100
Bill: Rs 50.00
```

Alternative Concept – fgets() + sscanf()/strtol()

Read each line into char buf[64] with fgets, then parse with sscanf or strtol (which reports the end pointer and ERANGE). Time: O(L) to read and O(L) to parse. Space: a fixed buffer of L_max+1 bytes (64 bytes) instead of none. Recovery: with scanf, a bad token stays in the input stream and must be flushed manually; with fgets the whole line is already consumed, so the next line starts clean, and the line can be reported in the error message. fgets+strtol is more robust.

Result: Correct specifiers, return-value checking and Rs/paise printing.